Design and Analysis of Algorithms 25/03/2026

Lecture 18: Introduction to Complexity Theory

Instructor: Sourav Chakraborty *Scribe: Abdulla,Rakesh Mandal*

1. A Motivating Problem List

Consider the following problems:

1. Sort an array.
2. Find the Minimum Spanning Tree (MST) of a given graph.
3. Is there a perfect matching in a given graph ?
4. Is p a prime number?
5. Does n have a factor $\leq k$?
6. Find the minimum vertex cover of a given graph.
7. Find a solution to a given Integer Linear Program (ILP).
8. Does the graph G have a Hamiltonian path?
9. Count the number of perfect matchings in G.
10. Does program P halt on input w ? (Halting Problem)
11. From a given chess position, does Black have a winning strategy?
12. Do two programs produce the same output on all inputs?
13. Can n points be clustered onto k clusters?

14. Does a given set of constraints have a solution?

How can we characterise these problems?

2. Classifying Problems

Computational problems can be broadly classified into three categories:

- **Decision Problems (YES/NO):** The answer is either YES or NO.
- **Optimization Problems:** Seek the best solution under some criterion (minimize / maximize), subject to satisfying constraints.
- **Counting Problems:** Count the number of valid solutions.

Observation:

Many of the problems in the list above can be converted into a corresponding decision problem. For example:

- *Find the MST* (optimization) $\longrightarrow$ *Does a spanning tree of weight* $\leq W$ *exist?* (decision). The optimization version can then be solved using $\mathcal{O}(\log n)$ calls to the decision oracle (e.g., binary search on the answer).
- *Factorise* n (optimization/search) $\longrightarrow$ *Does* n *have a factor* $\leq k$*?* (decision).

Note, however, that not every optimization problem can be reduced to a decision problem of this form in general.

Decision problems are the central problems in complexity theory, as they provide a clean, uniform framework for classification.

3. Formal Definitions: Decision Problems and Time Complexity

3.1 Decision Problems

Definition 1 (Decision Problem)**.** *A* ***decision problem*** *asks whether a given input x belongs to a language $L \subseteq \{0,1\}^*$.*

Formally, a decision problem is a function

$$f : \{0,1\}^* \to \{0,1\}$$

satisfying

$$f(x) = \begin{cases} 1 & \text{if } x \in L, \\ 0 & \text{if } x \notin L, \end{cases} \qquad \text{for all } x \in \{0,1\}^*.$$

The input x is encoded as a binary string, and its size is $n = |x|$.

3.2 Time Complexity of an Algorithm

Definition 2 (Worst-case Time Complexity)**.** *For an algorithm A, the* ***worst-case time complexity*** *$t(n)$ is the maximum number of steps A takes over all inputs of size n:*

$$t(n) \;=\; \max_{x \in \{0,1\}^n} \big\{ \#\ \text{steps of } A(x) \big\}.$$

Here $n = |x|$ is the length of the binary representation of the input.

3.3 Polynomial and Exponential Time

- **Polynomial Time.** If $t(n) = \mathcal{O}(n^c)$ for some constant c, algorithm A runs in *polynomial time*. This is considered efficient.
- **Exponential Time.** If $t(n) = \mathcal{O}(c^n)$ for some constant $c > 1$, algorithm A runs in *exponential time*. This is generally infeasible for large inputs.

Note:

- **Linear Programming (LP) $\in$ P** — the Simplex method is not polynomial in the worst case, but the Ellipsoid and Interior Point methods achieve polynomial time.
- **PRIMES $\in$ P** — long believed to require super-polynomial time, resolved by the AKS algorithm.

4. Complexity Classes: P and NP

4.1 The Class P

Definition 3. **P** *is the class of all decision problems for which there exists a* deterministic polynomial-time *algorithm:*

$$\mathbf{P} \;=\; \big\{\, L \;\big|\; \exists \text{ a poly-time algorithm } A \text{ that decides } L \,\big\}.$$

4.2 The Class NP

NP captures problems with *non-deterministic* polynomial-time solutions — informally, problems where one can **guess** a certificate and verify it efficiently.

Definition 4 (**NP**). ***NP*** *is the class of decision problems for which a proposed solution (certificate) can be* ***verified*** *in polynomial time by a deterministic algorithm.*

4.3 The Class co-NP

Definition 5 (**co-NP**). ***co-NP*** *is the complement class of* ***NP****: the* NO*-instances can be verified in polynomial time.*

4.4 Complexity Hierarchy

The known containment relationships are:

$$\mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{EXP}$$

Class	**Description**
P	Problems solvable in deterministic polynomial time.
NP	Problems verifiable in polynomial time (YES-instances).
$\mathsf{co-NP}$	Complement of NP; NO-instances verifiable in polynomial time.
EXP	Problems solvable in exponential time $\mathcal{O}(2^{\mathrm{poly}(n)})$.

The central open question in computer science:

$$\mathbf{P} \stackrel{?}{=} \mathbf{NP}$$

5. NP-Completeness

Definition 6 (NP-Complete)**.** *A problem L is **NP-Complete** if:*

1. $L \in \mathsf{NP}$ *(solutions are verifiable in polynomial time), and*
2. L *is **NP-Hard**: every problem in* NP *reduces to L in polynomial time.*

The canonical NP-Complete problem is **SAT** (Boolean Satisfiability): given a Boolean formula, determine whether there is a satisfying assignment.

6. Computability Theory

6.1 Historical Context: Hilbert and Gödel

In 1896, Hilbert posed a famous list of problems, one of which asked whether every mathematical statement is decidable (computable). Many of these problems remain open today.

Gödel (1931) proved that not every mathematical problem is computable:

Theorem 6.1 (Gödel's Incompleteness Theorem)**.** *In any consistent, sufficiently powerful formal system, there exist true statements that can* neither be proved nor disproved *within the system.*

This result — primarily about logic — established fundamental limits of formal axiomatic systems.

6.2 What is Computable?

Church and **Turing** independently formalised computability in the 1930s. Turing's model — the *Turing Machine* — convinced Gödel and the broader community.

- **Church–Turing Thesis:** Any effectively computable function can be computed by a Turing Machine.

6.3 The Halting Problem

Definition 7 (Halting Problem)**.** *Given a program P and an input w, does P halt on w?*

Theorem 6.2 (Turing, 1936)**.** *The Halting Problem is* ***undecidable****: no algorithm can decide it for all inputs.*

The proof uses a *diagonalisation* argument, and is deeply connected to Gödel's incompleteness theorem via the Church–Turing thesis.